Choosing the **x86 Intel SGX approach** is the fastest path to a working prototype because the authors of the *Secure IIoT Gateway Architecture* paper built and evaluated their prototype using the **Intel SGX SDK** and the **MbedTLS** library. Better yet, the official Intel SGX SDK includes a built-in **Simulation Mode** (SGX_MODE=SIM), meaning you can write, compile, and run the exact same Enclave code (ECALLs and OCALLs) on **any standard x86_64 Linux PC or VM**, even if your CPU does not have physical SGX hardware.

1. Requirements to Start Development

To replicate the paper's prototype environment without building low-level OS components from scratch, you need the following development stack:

Hardware & OS Requirements

- **Machine:** Any standard x86_64 PC, laptop, or Virtual Machine (the paper used an Intel Core i7-8750H, but *any* x86_64 CPU works when compiling in SGX Simulation Mode).
- **Operating System:** Ubuntu 20.04 LTS, 22.04 LTS, or 24.04 LTS (64-bit).

Software & Library Requirements

1. **Intel SGX SDK for Linux (linux-sgx):**
    - Provides the compiler toolchain, the **sgx_edger8r** tool (which automatically generates your cross-world ECALL and OCALL C stubs), the trusted C/C++ standard libraries (tlibc/tlibcxx), the trusted crypto library (sgx_tcrypto), and the **SGX Simulation Mode** runtime (libsgx_urts_sim.so / libsgx_trts_sim.so).
2. **SGX-Compatible MbedTLS (MBEDTLS):**
    - The paper uses the **MbedTLS** library inside the Secure World to manage TLS/HTTPS sessions and execute internal OCALLs/ECALLs for network I/O. You can use **mbedtls-SGX** (or Intel's official intel-sgx-ssl / mbedtls enclave port) so you don't have to port TLS socket wrappers yourself.
3. **Cryptographic Primitives (Inside the Enclave):**
    - **AES (128-bit)** and **SHA-256**: Provided out-of-the-box by both MbedTLS and Intel's trusted sgx_tcrypto library.
    - **ECDH (SECP256k1 curve)**: Provided natively by MbedTLS (MBEDTLS_ECP_DP_SECP256K1_ENABLED).
    - **Poly1305-AES MAC**: Provided by MbedTLS (mbedtls_poly1305_mac) or a lightweight single-file C reference implementation (poly1305-donna) compiled into the enclave.
4. **Time Synchronization (Optional for Initial Testing):**
    - **linuxptp (ptp4l)**: For IEEE 1588 Precision Time Protocol (PTP) synchronization. During initial local simulation on a single PC, you can read the system monotonic/real-time clock directly and pass the timestamp t into the enclave.

2. Project Directory Structure

When using the Intel SGX SDK, your RSU Gateway project is cleanly divided into two folders

that compile into two binaries—matching **Figure 3** and **Figure 5** of the paper:

Plaintext
```
rsu-sgx-gateway/
├── Makefile                # Compiles with SGX_MODE=SIM (no SGX hardware needed)
├── Enclave/                # SECURE WORLD (Compiled into enclave.signed.so)
│   ├── Enclave.edl         # Defines ECALLs (into TEE) and OCALLs (out to Linux)
│   ├── Enclave.config.xml  # Configures Enclave Heap/Stack size
│   ├── Enclave_private.pem # Signing key for the enclave binary
│   ├── control.cpp         # Control component (Bootstrap, ECDH, OTP, Key DB)
│   └── marshaling.cpp      # Secure Marshaling (AES decrypt, Poly1305, JSON, MbedTLS)
└── App/                    # NON-SECURE WORLD (Compiled into normal Linux binary 'app')
    ├── main.cpp            # Initializes the SGX Enclave
    ├── packet_queue.cpp    # OT Queue (from V2X/OBUs) & IT Queue (to Cloud/ESA)
    └── packet_manager.cpp  # Dispatches packets into Enclave via ECALLs & handles
OCALLs
```

3. Step-by-Step Development Workflow

Step 1: Install the Intel SGX SDK (Simulation Mode Only)

You only need the **Intel SGX SDK** installer (you do **not** need the SGX kernel driver or PSW daemon when running in Simulation Mode).

1. Install build dependencies on Ubuntu:
   Bash
   ```
   sudo apt update
   sudo apt install build-essential ocaml ocamlbuild automake autoconf libtool wget
   python-is-python3 libssl-dev git cmake
   ```

2. Download and install the official Intel SGX SDK Linux .bin installer from Intel's repository, then source the environment variables:
   Bash
   ```
   chmod +x sgx_linux_x64_sdk_*.bin
   ./sgx_linux_x64_sdk_*.bin --prefix=/opt/intel
   source /opt/intel/sgxsdk/environment
   ```

3. Verify your setup by compiling one of the SDK's included sample projects in Simulation Mode:
   Bash
   ```
   cd /opt/intel/sgxsdk/SampleCode/SampleEnclave
   ```

```
make SGX_MODE=SIM
./app
```

Step 2: Define the Boundary Interface (Enclave.edl)

Instead of writing kernel modules or SMC assembly, you define the functions that cross between the **Non-Secure World** (App) and **Secure World** (Enclave) inside Enclave.edl using the SGX Enclave Definition Language. The sgx_edger8r tool automatically generates the C wrapper code for you.
Create Enclave/Enclave.edl with the entry points required by **Figure 3** and **Figure 5**:

C

```
enclave {
    from "sgx_tstdc.edl" import *;

    trusted {
        /* ECALL 1: Gateway initialization & Root of Trust setup */
        public int ecall_init_gateway(
            [in, size=cert_len] const uint8_t* gw_cert, size_t cert_len,
            [in, size=esa_len] const uint8_t* esa_pub_key, size_t esa_len
        );

        /* ECALL 2: Control Component - Register Activation_Token from ESA */
        public int ecall_control_add_activation_token(
            uint64_t t0, uint64_t tf,
            [in, size=16] const uint8_t* auth_d,
            [in, size=32] const uint8_t* auth_eg
        );

        /* ECALL 3: Control Component - Handle V2X Node Bootstrap (ECDH + Auth_Request) */
        public int ecall_control_handle_bootstrap(
            int msg_type,
            [in, size=in_len] const uint8_t* in_pkt, size_t in_len,
            [out, size=out_max_len] uint8_t* out_pkt, size_t out_max_len,
            [out] size_t* actual_out_len,
            uint64_t current_ptp_time
        );

        /* ECALL 4: Secure Marshaling - OT Queue -> Decrypt/Verify -> JSON -> Encrypt -> IT
Queue */
        public int ecall_secure_marshal_ot_to_it(
```

```
        [in, size=ot_pkt_len] const uint8_t* ot_pkt, size_t ot_pkt_len,
        uint64_t current_ptp_time,
        [out, size=it_buf_max] uint8_t* it_https_pkt, size_t it_buf_max,
        [out] size_t* it_pkt_len
    );
  };

  untrusted {
    /* OCALL 1: Push encrypted HTTPS packet to Non-Secure IT Queue / Socket */
    int ocall_net_send([in, size=len] const uint8_t* buf, size_t len);

    /* OCALL 2: Read encrypted HTTPS response from Non-Secure IT Queue / Socket */
    int ocall_net_recv([out, size=len] uint8_t* buf, size_t len);

    /* OCALL 3: Save/Load sealed keys (K_dg, K_G, Auth_d) to Non-Secure disk for Reboot
Recovery */
    int ocall_save_sealed_state([in, size=len] const uint8_t* sealed_blob, size_t len);
    int ocall_load_sealed_state([out, size=max_len] uint8_t* sealed_blob, size_t max_len);
  };
};
```

Step 3: Code the Non-Secure Host Application (App/)

In the App/ folder, write standard Linux C++ code for the **Packet Queue** and **Packet Manager**:

1. **Initialize the Enclave (main.cpp):**
    - Call sgx_create_enclave("enclave.signed.so", SGX_DEBUG_FLAG, ...) to create the isolated Secure World memory space.
    - Call ecall_init_gateway(...) to load pre-configured security parameters (such as the gateway certificate CY_g and ESA public key CY_e—note that in the paper's benchmark prototype, security parameters were pre-configured prior to deployment).
2. **Implement the OT Queue and IT Queue (packet_queue.cpp):**
    - Use standard C++ std::queue<std::vector<uint8_t>> protected by std::mutex.
    - Open a UDP socket (representing the local V2X/OT network interface) that pushes incoming raw byte frames into the **OT Queue**.
    - Open a TCP/HTTPS client socket (or local loopback socket to a mock Cloud/ESA server) that transmits encrypted packets pulled from the **IT Queue**.
3. **Implement the Packet Manager (packet_manager.cpp):**
    - Poll the OT Queue. Inspect the packet header:
        - If it is a bootstrap packet (ECDH_Request, Auth_Request, or Active), call ecall_control_handle_bootstrap(...).
        - If it is a regular data packet formatted as

$Message(AES(K_G, Data), Auth_d, OTP_{d_t})$, call ecall_secure_marshal_ot_to_it(...) and push the returned encrypted HTTPS payload into the IT Queue.

Step 4: Code the Enclave Secure Marshaling Module (Enclave/marshaling.cpp)

To replicate the paper's exact prototype first (Section 6 of the paper), implement ecall_secure_marshal_ot_to_it before coding the complex bootstrap state machine. As the paper notes, you can initially pre-configure the keys (K_G, K_{dg}, $Auth_d$, and K_{eg}) inside the enclave to test the data pipeline immediately:

1. **Unpack the Incoming OT Packet:**
   - Split the incoming byte buffer ot_pkt into three fields: the encrypted payload $AES(K_G, Data)$, the 16-byte device token $Auth_d$, and the 16-byte MAC OTP_{d_t}.
2. **Lookup Keys in the OT Key Manager:**
   - Use $Auth_d$ as the key in an in-enclave std::map to retrieve that vehicle/node's individual secret K_{dgi} and its group key K_{Gi}.
3. **Authenticate via Poly1305-AES:**
   - Compute $Poly1305(K_{dgi}, (Auth_d \oplus t), m)$ inside the enclave and compare it against the packet's OTP_{d_t} (allowing timestamp $t \pm 1$). If verification fails, abort and return an error code.
4. **Decrypt SmartData Payload via AES-128:**
   - Decrypt $AES(K_{Gi}, Data)$ in enclave RAM using sgx_rijndael128GCM_decrypt or mbedtls_aes_crypt_cbc / ecb to recover the clear-text SmartData struct (location, timing, metadata, value, Auth_d).
5. **Marshal to JSON:**
   - Format the decrypted fields into a JSON string inside enclave memory using snprintf:
     C
     snprintf(json_buf, sizeof(json_buf),

     "{\"Location\":%u,\"Timing\":%llu,\"Metadata\":%u,\"Value\":%d,\"Trust\":\"%s\"}",
     sd->location, sd->timing, sd->metadata, sd->value, auth_d_hex);
6. **Encrypt for the Cloud (MBEDTLS / AES):**
   - Pass json_buf into your enclave's **MbedTLS** session (or encrypt with session key K_{eg} and prepend the HTTPS POST header, as shown in **Figure 5**), and copy the encrypted result into it_https_pkt (or send it directly via MbedTLS's internal ocall_net_send callback).

Step 5: Code the Enclave Control Module (Enclave/control.cpp)

Once your Secure Marshaling pipeline works with pre-configured keys, add dynamic vehicle/OBU onboarding by implementing the 12-step bootstrap protocol from **Figure 4**:

1. **Pending Activation Store:** Store $(t_0, t_f, Auth_d, Auth_{eg})$ structs inside enclave RAM when ecall_control_add_activation_token is called.
2. **ECDH Handler (Arrows 4 & 5):** Use MbedTLS's SECP256k1 functions (mbedtls_ecdh_gen_public / mbedtls_ecdh_compute_shared) to receive Y_d, output Y_g, and derive K_{dg}.
3. **OTP Verification & Group Key Provisioning (Arrows 6, 7 & 8):** Verify Auth_Request(OTP_{d_t}) against pending tokens. On match, use Intel SGX's sgx_seal_data() API to encrypt the state $(Auth_{eg}, Auth_d, K_{dg}, K_G, CY_e)$ and pass it to ocall_save_sealed_state() so it persists safely on disk across gateway reboots, then return Auth_Granted($Auth_{eg}, OTP_{d_t}, K_G, CY_e$) encrypted with K_{dg}.

Step 6: Build, Test, and Benchmark Locally

1. Compile your project in **SGX Simulation Mode**:
   Bash
   make SGX_MODE=SIM SGX_DEBUG=1

2. Run a mock Python or C script on your machine that acts as a V2X On-Board Unit (OBU)—sending encrypted SmartData UDP packets (from 32 bytes up to 64 KB) to your gateway's OT Queue—and another script acting as the Cloud HTTPS server receiving the JSON POST requests from the IT Queue.
3. Measure the turn-around time with std::chrono::high_resolution_clock around ecall_secure_marshal_ot_to_it to compare your RSU's marshaling latency directly against **Figure 6** of the paper (18.4 μs for 32B up to 4307.4 μs for 64KB).